performance? We formally represent the change as $\boldsymbol{W}' = \boldsymbol{W} + \Delta\boldsymbol{W}_{\text{edit}}$, where $\boldsymbol{W}$ is the original weight matrix, and $\Delta\boldsymbol{W}_{\text{edit}}$ represents the modifications made during editing. Therefore, our primary focus in this section is to discern the differences between the matrices $\Delta\boldsymbol{W}_{\text{edit}}$ for different editing methods.

Sparsity An important characteristic of knowledge editing is its intention to modify a specific piece of knowledge within the model. This suggests an intuitive hypothesis that the $\Delta\boldsymbol{W}$ matrix is likely to be sparse. Following the approach of De Cao et al. [168], we present visualizations that capture weight updates resulting from knowledge edits, as depicted in Figure 6.

ROME, MEND, and MEMIT exhibit a distinct pattern of sparse updates, while fine-tuning spreads its modifications more uniformly across weights. Particularly, for knowledge editing methods like ROME and MEMIT, it is intriguing to observe a concentrated focus on one or several columns of the value layer. This finding aligns with earlier research that emphasizes the value layer's pivotal role in encapsulating correlated knowledge [42]. Regarding the MEND methods, we propose that the learned hypernetwork can be viewed as a tool or a "probe" that helps us explore and understand the internal mechanisms used by the model to encode knowledge, providing insights into how the model represents and processes information.

Mapping to Embedding Space To further investigate the differences between different editing methods, we conduct an embedding space analysis following the approach of Dar et al. [208]. They analyze the Transformer's parameters by mapping the weights of the LLMs to the vocabulary space and find that the embedding space can interpret these weights. Here, we map the two matrices, $\boldsymbol{W}'$ and $\boldsymbol{W}$, to observe the differences between these methods. From the sparsity analysis, we select the top five columns of the updated value matrix $\Delta\boldsymbol{W}$ and map the corresponding columns of $\boldsymbol{W}'$ and $\boldsymbol{W}$ into the embedding matrices $\boldsymbol{E}$ to obtain the logits in the vocabulary space. We then compute the Hit@10 and Hit@50 of the new knowledge in the output logits. We select cases from ZsRE where all four methods successfully edit the knowledge and present the average performance in Figure 7. From the figure, we observe that MEND and MEMIT significantly inject the target knowledge into the parameters. Notably, MEND demonstrates a remarkable capacity for editing, with the Hit@50 rate already exceeding 90% before the edit. This means that MEND might be able to find and change the right neurons that hold the target knowledge without having to do a full knowledge-locating analysis. After the editing process, we observe a substantial increase in the Hit@10 score. In fact, in our experiments, the Hit@1 for MEND is also above 90% after editing, demonstrating its strong editing capacity. For MEMIT, we also observe an increase in Hit@50 (59.7% $\rightarrow$ 70.2%), and the original neurons already have a high Hit score before editing. However, for ROME and FT-L, we do not observe an increase in performance, indicating that their editing mechanisms require further investigation to understand their specific characteristics and limitations.

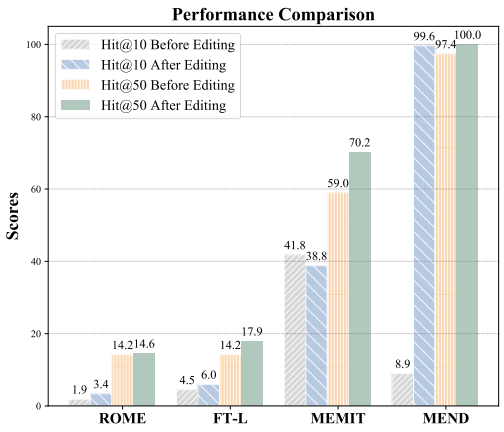


Figure 7: The Hit@10 and Hit@50 performance for the target knowledge in the model's parameters before and after editing.

5.2 Analysis of Knowledge Locating

As we have discussed in the previous part, the knowledge stored in LLMs is not structured. Also, in the previous experiments, we found that the performance of current editing in terms of portability is not good. As previous works have found [69, 155, 157], editing factual knowledge does not necessarily enable models to utilize it during reasoning and application. Meanwhile, Hase et al. [209] found edit success unrelated to where facts are stored, as measured by causal tracing. These works highlight that current editing methods are insufficient and pose skepticism against the effectiveness of current knowledge location analysis. Chang et al. [210] introduces two benchmarks: **INJ** and **DEL** to investigate "Do any localization methods actually localize memorized data in LLMs?". They conduct experiments on current

localization methods, including zero-out and integrated gradients, and proposed two prune-based localization methods: SLIMMING and HARD CONCRETE. Two benchmarks show positively correlated results and demonstrate strong localization abilities of integrated gradients, SLIMMING, and HARD CONCRETE. At the same time, the DEL Benchmark shows that all methods struggle to balance between erasing the target sequence and retaining other memorized data; in other words, the neurons identified by localization methods tend to also be relevant for memorizing some other sequences. Additionally, Ju and Zhang [211] proposed a benchmark for assessing the effectiveness of current knowledge location methods and three evaluation metrics: *consistency, relevance, and unbiasedness*. This benchmark plays a crucial role in facilitating a comprehensive evaluation of whether current locating methods can accurately pinpoint model parameters associated with specific factual knowledge. Here, we make a simple analysis of the location methods for knowledge editing based on the benchmark. We adopt the computing of the **Relative Similarity** (RSim) as: $\max\left(\frac{\text{Sim}_{\text{cand}} - \text{Sim}_{\text{all}}}{1 - \text{Sim}_{\text{all}}}, 0\right)$.

We adopt their dataset klob-r (designed for measuring consistency) and klob-c (designed for measuring relevance) and apply them to the casual analysis method proposed by ROME [79]. Since the casual analysis is a layer-wise intervention, here we compute the similarity using the overlap between the identified layers. We show the RSim score in Figure 8. Here, we can find the Rsim score is less than 0.6 when we consider more than five layers for both consistency and relevance, which means the locating results for unrelated knowledge and related knowledge chains didn't show much difference. To be more tangible, we conduct a case study here.

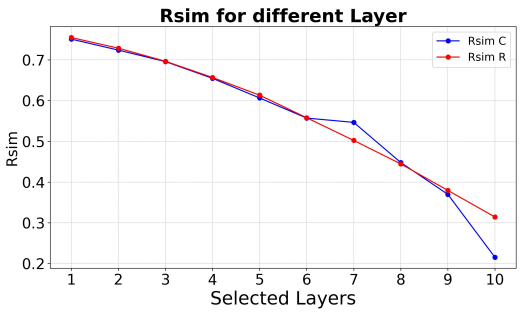
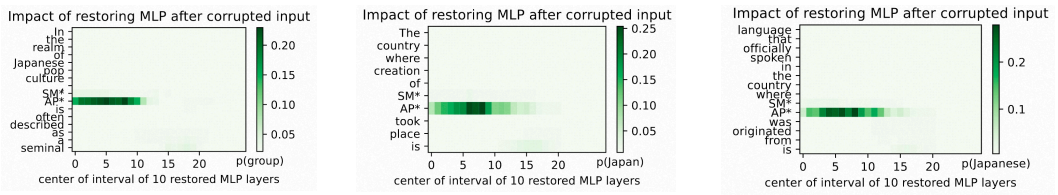


Figure 8: RSim for the different number of layers.

Case Study We consider three settings for a given fact associated with the entity *SMAP* and show it in Figure 9. We first conduct a causal analysis of the fact: [SMAP $\xrightarrow{\text{created in}}$ Japan]. Then, we consider a related question with the fact [SMAP $\xrightarrow{\text{created in}}$ Japan $\xrightarrow{\text{language}}$ Japanese], where the model should answer the question based on the fact. Finally, we adopt an unrelated fact [SMAP $\xrightarrow{\text{type of}}$ seminal group] with the question. The results show that these facts are possibly related to the same place around 5 layers. However, as Ju and Zhang [211] mentioned, **the locating results for specific knowledge and its related knowledge chain should exhibit greater similarity compared to those for unrelated knowledge.** Currently, casual analysis methods seem to just locate the area that is related to the entity itself, not the whole fact. Whether the model performs these answers by cheating with answers memorized from the pretraining corpus or via a multi-step reasoning mechanism is still unclear. This is strongly related to the knowledge editing tasks. More broadly, better insight into models' knowledge processes could unlock capabilities like explainability and fact verification. However, fully understanding how exactly knowledge is organized and interconnected within such large models presents an ongoing challenge. Key open questions include developing methods to trace factual usage during reasoning, designing location techniques that identify knowledge most salient for model outputs, and learning how architectural properties relate to knowledge utilization. Unpacking these knowledge architectures will be integral to enabling more precise and robust model interventions through approaches like knowledge editing but currently manipulating only the MLP weights is not enough.

5.3 The Implicit Knowledge Structure in LLMs

Understanding the knowledge structure in LLM is crucial for effective knowledge editing. Previous research often conceptualized knowledge within LLMs as resembling triples in Knowledge Graphs (KG), comprising subjects, relations, and objects. This analogy, while useful, simplifies the intricate nature of knowledge representation in LLMs.



(a) (b) (c)

Figure 9: First, we conduct a causal analysis of the fact with the entity [SMAP $\xrightarrow{\text{created in}}$ Japan]. Second, we consider a related question with the fact,[SMAP $\xrightarrow{\text{created in}}$ Japan $\xrightarrow{\text{language}}$ Japanese], where the model should answer the question based on the fact. Then, we adopt an unrelated fact [SMAP $\xrightarrow{\text{type of}}$ seminal group].

Editing knowledge in a KG, where the task usually involves modifying a single relationship between two nodes, is comparatively straightforward. KGs inherently support easy reasoning tasks and allow for the preservation of the rest of the knowledge structure. This resilience is illustrated in Figure 10, where edits and subsequent recovery processes result in the complete restoration of the original KG structure. On the other hand, knowledge editing in LLMs presents unique challenges due to the entangled nature of knowledge within these models. Unlike KGs, where knowledge is neatly compartmentalized, in LLMs, knowledge is distributed across various parameters and layers, making it difficult to isolate and edit specific information without affecting other knowledge areas. The current perspective of viewing knowledge in LLMs as triples is somewhat limited and fails to capture the full complexity and interconnected nature of these models. This complexity is further highlighted by previous work [184, 101], who discuss the challenges of modifying intrinsic knowledge within parameters.

Furthermore, previous research has revealed that knowledge editing in LLMs can lead to unintended propagation effects. Li et al. [206] illustrates that current knowledge editing methods can result in **knowledge conflict** and **knowledge distortion** within LLMs. Unlike structured knowledge bases, neural networks lack strict constraints on knowledge structure and interrelationships. This makes it difficult to confine edits to a localized scope within the model, and the free-form nature of LLMs further complicates the editing process. Consequently, a more comprehensive understanding of the LM's mechanisms is required.

Currently, methods like T-Patcher or IKE offer **plug-and-play functionality and easy reversibility**. They provide flexibility and user-friendliness and can be easily integrated into or detached from the LLMs as needed. These methods aim to mitigate some of the challenges associated with knowledge editing in LLMs, allowing for convenient and reversible modifications. As the field evolves, it is imperative to continue developing methods that not only address the challenges of knowledge editing but also harness the full potential of these complex systems, turning vanilla LLMs into **WikiModels**, a.k.a., neural knowledge bases that is feasibility for editing.

6 Applications

In this Section, we will summarize recent approaches that utilizes knowledge editing techniques for various applications and illustrate potential directions for future exploration.

6.1 Efficient Machine Learning

Model Updating While knowledge editing techniques directly modify or augment model parameters, realizing their full potential requires translating these internal updates into LLMs for downstream tasks. Recent research has explored integrating knowledge editing into various tasks, including **question answering**, **fact checking**, and **natural language generation**. For question answering, approaches like MeLLo [155] decompose complex questions and iteratively retrieve and edit knowledge to arrive at multi-hop answers. Reckon [212] proposes a method to teach LLMs to reason by updating their parametric knowledge through back-propagation. This approach enables models to